

Compte-rendu

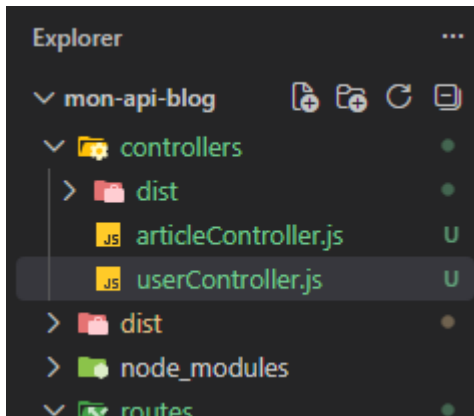
MERN

TP_2

Nom : adem boukadi Class : 5eme DS et IA G1

Exercice 1 :

En créer un nouvel fichier « userController.js » on « controllers »



En créer un tableau de 2 utilisateurs avec « id, name, email, rôle »

```
let users = [
  { id: 1, name: "Adem Dupont", email: "Adem@example.com", role: "admin" },
  { id: 2, name: "abdel", email: "abdel@example.com", role: "user" }
];
```

```
let prochainId = 3;
```

En implémenté « getAllUsers »

```
const getAllUsers = (req, res) => {
  const { author } = req.query;
  let resultat = users;

  if (author) {
    resultat = users.filter(u => u.author === author);
  }

  res.status(200).json({
    total: resultat.length,
    users: resultat
  });
};
```

En implémenté « getUsersById »

```

const getUserById = (req, res) => {
  const id = Number(req.params.id);
  const user = users.find(u => u.id === id);

  if (!user) {
    return res.status(404).json({ message: "Utilisateur n"
  }

  res.status(200).json(user);
};

```

En implémenté « createUser »

```

const createUser = (req, res) => {
  const { name, email, role } = req.body;
  if (!name || !email || !role) {
    return res.status(400).json({ message: "Tous les champs sont obligatoires" });
  }
  const nouvelUser = {
    id: prochainId++,
    name,
    email,
    role
  };
  users.push(nouvelUser);
  res.status(201).json(nouvelUser);
};

```

En implémenté « deleteUsers »

```

const deleteUser = (req, res) => {
  const id = Number(req.params.id);

  const userExiste = users.some(u => u.id === id);

  if (!userExiste) {
    return res.status(404).json({
      error: `Impossible de supprimer : utilisateur ${id} introuvable`
    });
  }

  // On filtre pour ne garder que les utilisateurs ayant un ID différent
  users = users.filter(u => u.id !== id);

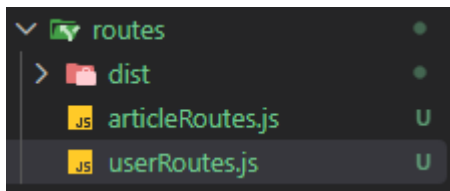
  res.status(201).json({
    message: 'Utilisateur supprimé'
  });
};

```

Export les 4 fonctions avec « module.exports »

```
module.exports = {  
  ...  
  getAllUsers,  
  getUserById,  
  createUser,  
  deleteUser  
}
```

Créer un fichier « userRoutes »



Instanciez un routeur avec « express.router() »

```
const express = require('express');  
const router = express.Router();
```

```
const { getAllUsers, getUserById, createUser, deleteUser } = require('../controllers/userController');
```

Associez les 4 verbes

```
// Association des 4 verbes HTTP  
router.get('/', getAllUsers);  
router.get('/:id', getUserById);  
router.post('/', createUser);  
router.delete('/:id', deleteUser);
```

Exporter le router

```
// Export du routeur  
module.exports = router;
```

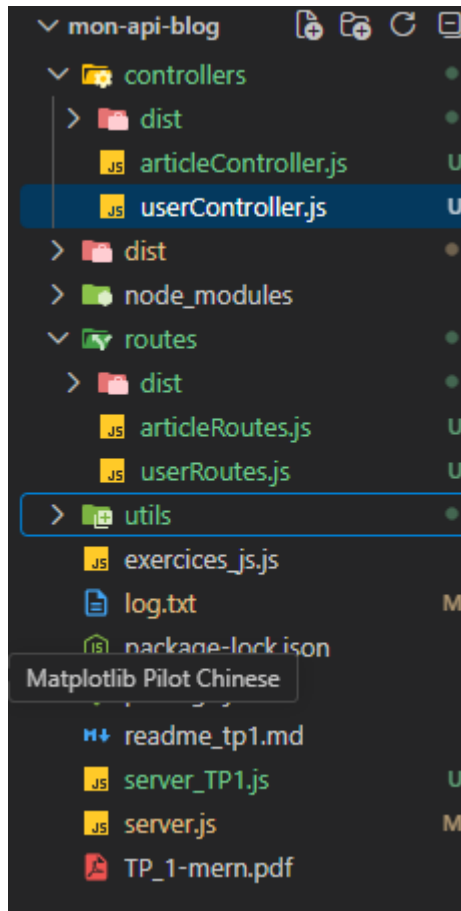
Montez le routeur sur le préfixe /api/users avec « app.use() »

```
// Montage du routeur sur le préfixe /api/users
app.use('/api/users', userRoutes);

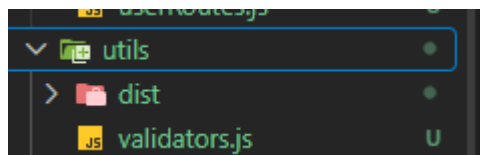
app.listen(PORT, () => {
  console.log(`Serveur démarré sur le port ${PORT}`);
});
app.listen(PORT, () => {
  console.log(`Serveur disponible sur http://localhost:
});
```

Exercice 2

Créées un dossier « utils »



Créez le fichier utils/validators.js



```
// utils/validators.js

const estNonVide = (chaine) =>
  | typeof chaine === 'string' && chaine.trim().length > 0;

const estEmailValide = (email) => {
  return (
    | typeof email === 'string' &&
    | email.includes('@') &&
    | email.includes('.')
  );
};

module.exports = { estNonVide, estEmailValide };

```

Importez ce module dans « contrtollres/userController.js »

```
const {estNonVide, estEmailValide} = require('../utils/validators');
```

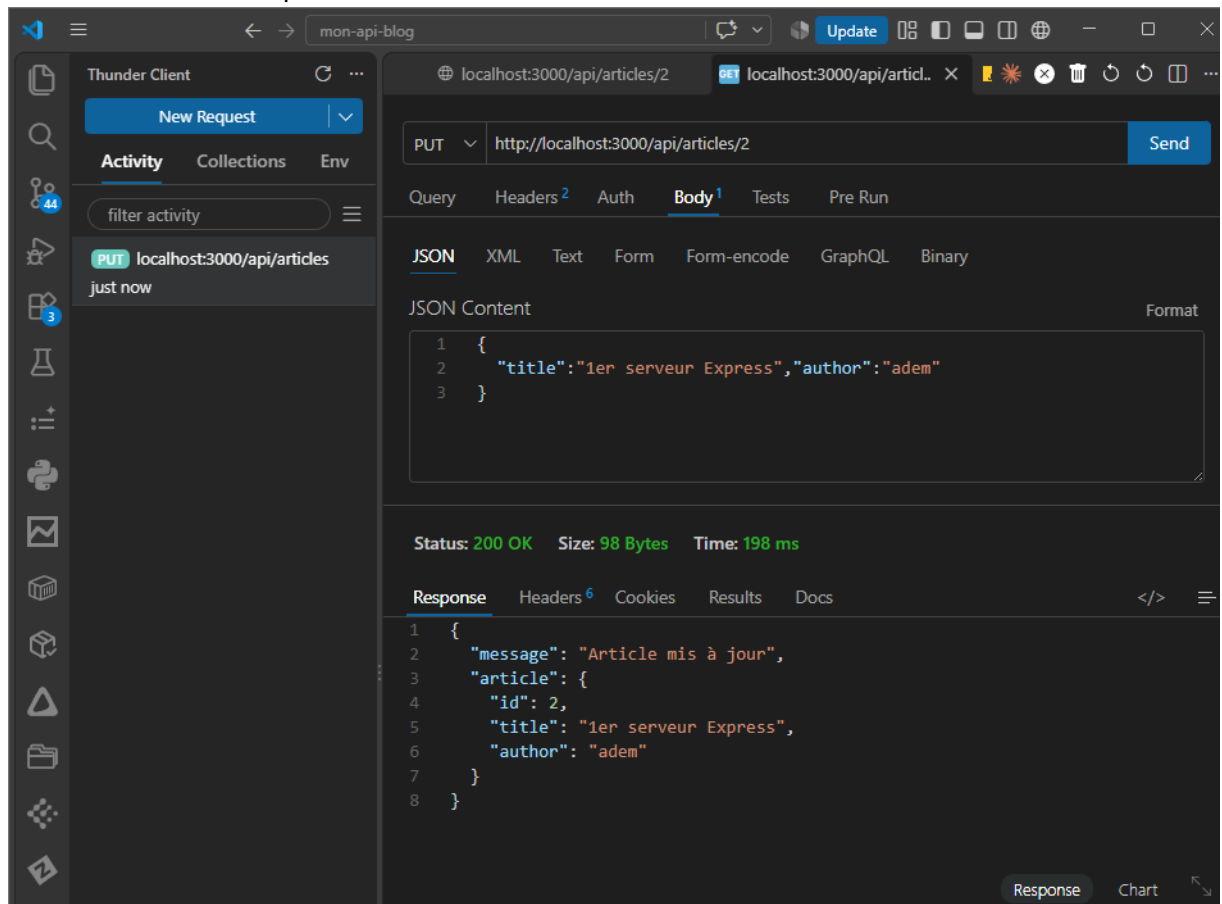
En utilise fonction dans « createUser »

```
const createUser = (req, res) => {
  const { name, email, role } = req.body;
  if (!estNonVide(name)) {
    return res.status(400).json({
      message: "Le nom est obligatoire et ne peut pas être vide."
    });
  }

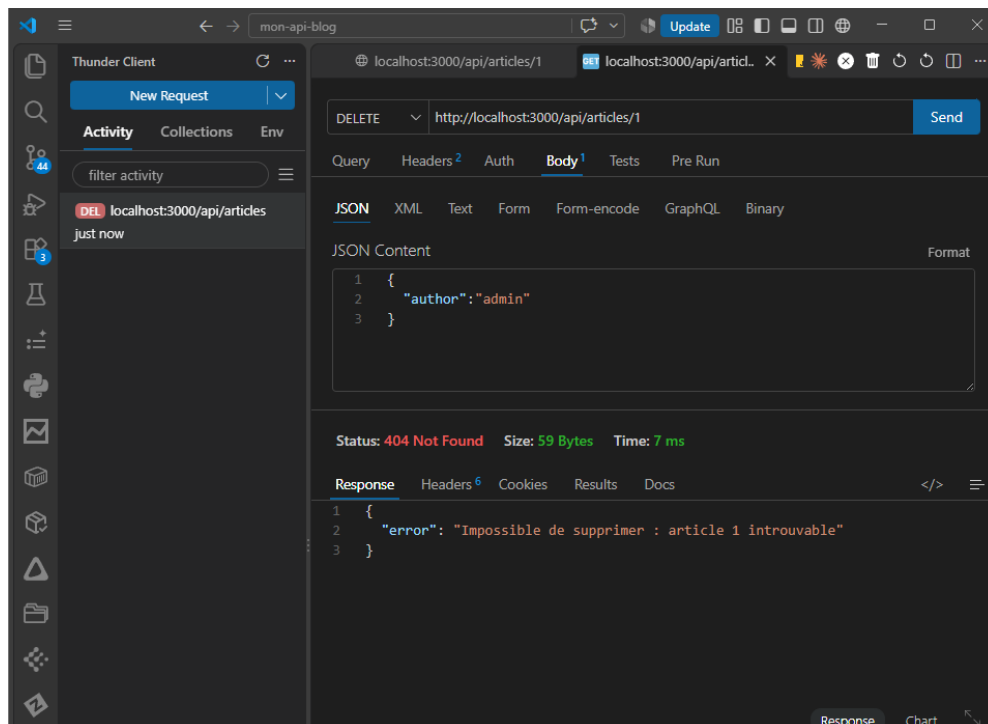
  // 2. Validation de l'email (doit contenir '@' et '.'
  if (!estEmailValide(email)) {
    return res.status(400).json({
      message: "L'email est invalide (doit contenir '@' et '.')."
    });
  }

  // 3. Validation du rôle (doit être renseigné)
  if (!estNonVide(role)) {
    return res.status(400).json({
      message: "Le rôle est obligatoire."
    });
  }
}
```

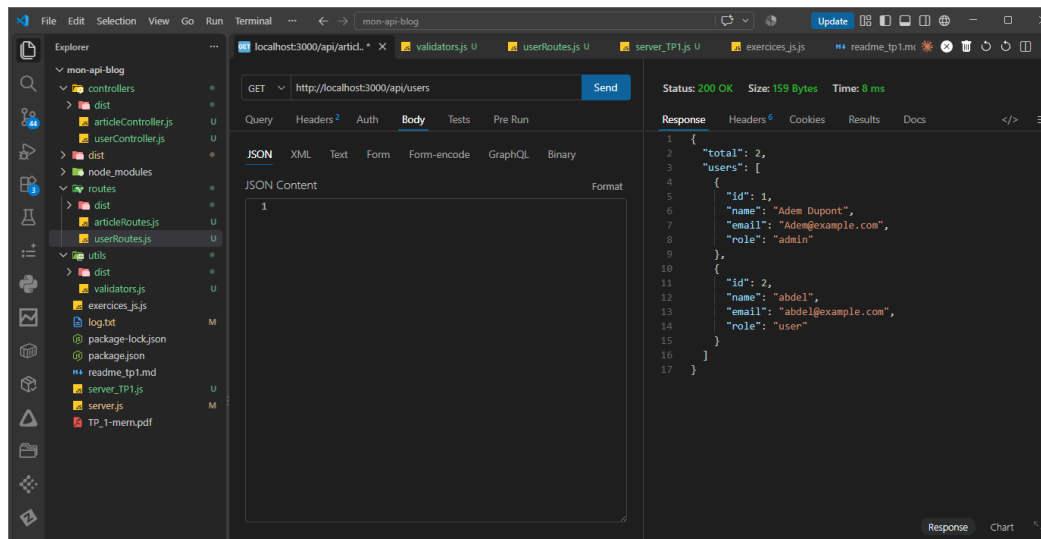
1.1 : test réussi de PUT /api/artucle/2



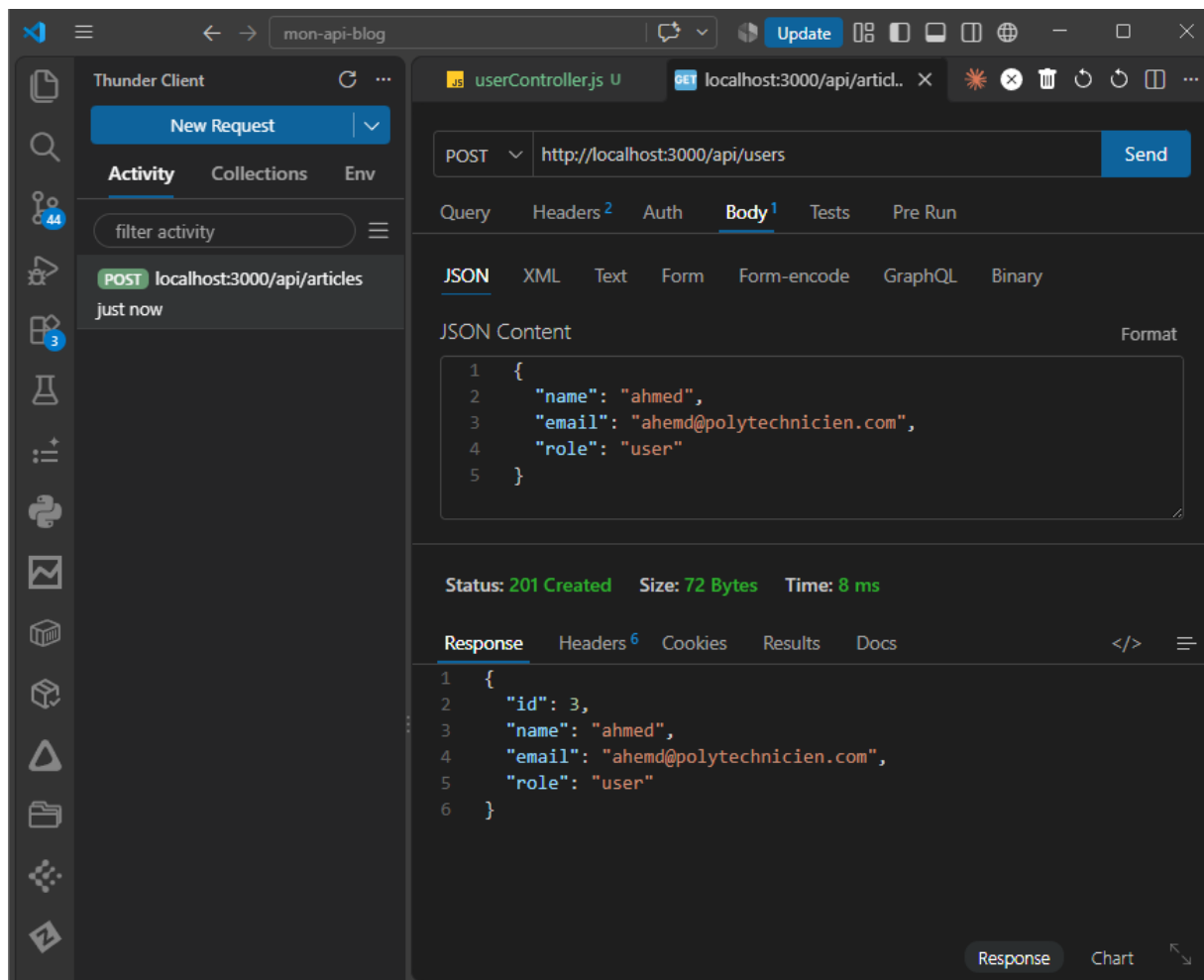
1.2. test réussi de DELETE /api/articles/



2.1 liste complète des users « GET »



2.2 201 created



2.3. bad request

The screenshot shows the Thunder Client interface. On the left, the 'Activity' panel shows a POST request to `localhost:3000/api/articles` labeled 'just now'. The main panel shows a POST request to `http://localhost:3000/api/users`. The request body is a JSON object:

```
{
  name: ahmed,
  email: ahemd@polytechnicien.com,
  role: user
}
```

The response status is **400 Bad Request** with a size of 1.21 KB and a time of 9 ms. The response body is an HTML error page:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <title>Error</title>
  </head>
  <body>
    <pre>SyntaxError: Unexpected token '&#39;&quot;&#39;, &quot;
    ;&quot;{\n  &nbsp;name&quot;... is not valid JSON<br>
    &nbsp; &nbsp;at JSON.parse (<anonymous>)<br> &nbsp;
    </pre>
```

Code defichers « userController.js »

```
// Initialisation du tableau users avec 2 utilisateurs fictifs (id, name, email, role)
```

```
let users = [
```

```
  { id: 1, name: "Adem Dupont", email: "Adem@example.com", role: "admin" },
```

```
  { id: 2, name: "abdel", email: "abdel@example.com", role: "user" }
```

```
];
```

```
let prochainId = 3;
```

```
const getAllUsers = (req, res) => {
```

```
  const { author } = req.query;
```

```
  let resultat = users;
```

```
  if (author) {
```

```
    resultat = users.filter(u => u.author === author);
```

```
  }
```

```
  res.status(200).json({
```

```
    total: resultat.length,
```

```
    users: resultat
```

```
  });
```

```
};
```

```
const getUserById = (req, res) => {
```

```
  const id = Number(req.params.id);
```

```
  const user = users.find(u => u.id === id);
```

```
  if (!user) {
```

```
    return res.status(404).json({ message: "Utilisateur non trouvé" });
```

```
  }
```

```
  res.status(200).json(user);
```

```
};  
const createUser = (req, res) => {  
  const { name, email, role } = req.body;  
  if (!estNonVide(name)) {  
    return res.status(400).json({  
      message: "Le nom est obligatoire et ne peut pas être vide."  
    });  
  }  
}
```

```
// 2. Validation de l'email (doit contenir '@' et '.')  
if (!estEmailValide(email)) {  
  return res.status(400).json({  
    message: "L'email est invalide (doit contenir '@' et '.')."  
  });  
}
```

```
// 3. Validation du rôle (doit être renseigné)  
if (!estNonVide(role)) {  
  return res.status(400).json({  
    message: "Le rôle est obligatoire."  
  });  
}
```

```
// Si tout est valide, on crée l'utilisateur  
const nouvelUser = {  
  id: prochainId++,  
  name,  
  email,  
  role  
};
```

```
users.push(nouvelUser);
res.status(201).json(nouvelUser);
};
const deleteUser = (req, res) => {
  const id = Number(req.params.id);

  const userExiste = users.some(u => u.id === id);

  if (!userExiste) {
    return res.status(404).json({
      error: `Impossible de supprimer : utilisateur ${id} introuvable`
    });
  }

  // On filtre pour ne garder que les utilisateurs ayant un ID différent
  users = users.filter(u => u.id !== id);

  res.status(201).json({
    message: 'Utilisateur supprimé'
  });
};
const {estNonVide, estEmailValide} = require('../utils/validators');

module.exports = {
  getAllUsers,
  getUserById,
  createUser,
  deleteUser
}
```

2. Code de fichier « userRoutes.js »

```
const express = require('express');  
  
const router = express.Router();  
  
const { getAllUsers, getUserById, createUser, deleteUser } =  
require('../controllers/userController');
```

```
// Association des 4 verbes HTTP
```

```
router.get('/', getAllUsers);  
  
router.get('/:id', getUserById);  
  
router.post('/', createUser);  
  
router.delete('/:id', deleteUser);
```

```
// Export du routeur
```

```
module.exports = router;
```

3. code de fichier « validators.js »

```
// utils/validators.js
```

```
const estNonVide = (chaine) =>  
  typeof chaine === 'string' && chaine.trim().length > 0;
```

```
const estEmailValide = (email) => {  
  return (  
    typeof email === 'string' &&  
    email.includes('@') &&  
    email.includes('.')  
  );  
};
```

```
module.exports = { estNonVide, estEmailValide };
```